

Machine Learning

From data to decision: a journey through learning

Mohammad Mahdi Nasimi

September 19, 2026

Contents

1	What is Machine Learning?	2
1.1	Opening Hook: Why Can't We Just Write the Rules?	2
1.2	Classical Software Engineering vs. Machine Learning	2
1.2.1	The Classical Approach	2
1.2.2	The Machine Learning Approach	2
1.2.3	A Concrete Example: Predicting House Prices	3
1.3	The Hypothesis Space: Where the Answer Lives	4
1.3.1	Three Examples	4
1.3.2	A Fundamental Tradeoff	4
1.4	From the Physical World to Raw Data	5
1.4.1	Data Is Always Imperfect	5
1.4.2	A Concrete Example	6
1.5	The Core Pipeline: Observation $\rightarrow$ Representation	6
1.5.1	Why Do We Need a Secondary Representation?	6
1.6	Geometric Intuition: Why Representations Matter	6
1.7	Practical Demonstration: Space Transformation in Python	7
1.8	A Short Taxonomy of Learning Paradigms	8
1.9	Summary and Key Takeaways	8
1.10	Before the Next Lecture	9

1 What is Machine Learning?

1.1 Opening Hook: Why Can't We Just Write the Rules?

Imagine you are asked to write a computer program that looks at a photograph and decides: “*Is there a cat in this image?*”

In classical software engineering, you would try to write explicit rules:

```
1 IF (has_pointy_ears) AND (has_whiskers) AND (has_fur) THEN "cat"
```

Listing 1: A hand-written rule attempt

But this fails almost immediately. Cats come in different colors, poses, sizes, and lighting conditions. They can be partially hidden. A stuffed toy might satisfy your rules. A small dog might violate them. The number of possible rules you would need to write — and the exceptions to those rules — grows without bound.

This is the fundamental problem that machine learning exists to solve.

Key Idea. Some problems are too complex, too high-dimensional, or too noisy to be solved by hand-written rules. Instead of writing the rules ourselves, we let the computer *learn* them from examples.

1.2 Classical Software Engineering vs. Machine Learning

1.2.1 The Classical Approach

In classical software engineering, a human expert encodes domain knowledge into a deterministic algorithm. The engineer designs a function f that maps inputs to outputs using explicit logic:

$$f : \mathcal{X} \rightarrow \mathcal{Y}, \quad y = f(x) \tag{1}$$

Here:

- $x \in \mathcal{X}$ is the input (a measurement, a sensor reading, a customer record).
- $y \in \mathcal{Y}$ is the output (a category, a number, a decision).

The engineer writes the rules by hand. The computer simply executes them.

$$\text{Classical Software Engineering: } (x, \text{Rules}) \xrightarrow{f} y \tag{2}$$

This works beautifully for well-defined problems: computing taxes, sorting a list, calculating a trajectory. But it fails for problems where the rules are unknown, too numerous, or too subtle to articulate.

1.2.2 The Machine Learning Approach

Machine learning *inverts* the process. Instead of writing the function f by hand, we:

1. Choose a family of candidate functions — called a *hypothesis space* $\mathcal{F} = \{f_\theta \mid \theta \in \Theta\}$, where θ is a vector of adjustable parameters.
2. Collect examples — a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ of input–output pairs.
3. Define a loss function $\mathcal{L}$ that measures how wrong a candidate function is.
4. Search for the parameters θ^* that make the function as correct as possible on the observed data.

$$\text{Machine Learning Framework: } (\mathcal{D}, \mathcal{F}, \mathcal{L}) \xrightarrow{\text{Optimizer}} \theta^* \implies f_{\theta^*}(x) \approx y \quad (3)$$

Analogy. Classical programming is like giving someone a recipe. Machine learning is like giving someone a kitchen, ingredients, and a way to measure whether the food tastes good — and letting them figure out the recipe themselves.

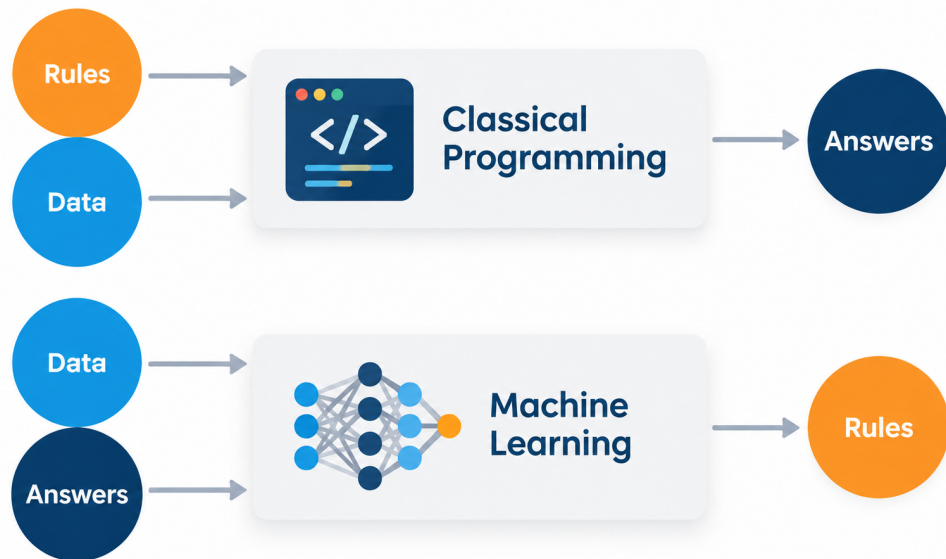


Figure 1: Classical Software Engineering vs. Machine Learning.

1.2.3 A Concrete Example: Predicting House Prices

Suppose we want to predict the price of a house from its size.

- **Input:** $x = \text{size in m}^2$.
- **Output:** $y = \text{price in euros}$.

Classical approach. A real estate expert writes a formula: $\text{price} = 3000 \times \text{size} + 50000$. Fixed. If the market changes, the expert must rewrite the formula.

Machine learning approach. We collect data on 1,000 houses — their sizes and their actual sale prices. We then choose a family of candidate formulas:

$$f_\theta(x) = \theta_1 \cdot x + \theta_0 \quad (4)$$

and search for the values of θ_0 and θ_1 that best fit the data. If the market changes, we simply re-run the search on new data.

This is the essence of *learning from data*: the rules are not written; they are discovered.

1.3 The Hypothesis Space: Where the Answer Lives

The most important design decision in machine learning is the choice of hypothesis space $\mathcal{F}$ — the family of functions we are willing to consider.

1.3.1 Three Examples

Example A — Linear hypothesis space.

$$\mathcal{F}_{\text{linear}} = \{f_{\theta}(x) = \theta_1 x + \theta_0 \mid \theta_0, \theta_1 \in \mathbb{R}\} \quad (5)$$

This is the family of all straight lines. It has only 2 parameters. Simple, interpretable, and easy to search.

Example B — Polynomial hypothesis space.

$$\mathcal{F}_{\text{poly}} = \left\{ f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k \mid \theta_0, \dots, \theta_K \in \mathbb{R} \right\} \quad (6)$$

This is the family of all polynomials of degree at most K . It has $K + 1$ parameters and can represent far more complex curves — but is harder to search.

Example C — Neural network hypothesis space.

$$\mathcal{F}_{\text{NN}} = \{f_{\theta}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2 \mid \theta = (W_1, b_1, W_2, b_2)\} \quad (7)$$

This family can represent extraordinarily complex functions. It may have millions or billions of parameters.

1.3.2 A Fundamental Tradeoff

Hypothesis space	Expressiveness	Search difficulty
Linear	Low	Easy
Polynomial (K large)	High	Moderate
Deep neural network	Very high	Hard

Table 1: Expressiveness versus search difficulty across hypothesis spaces.

Two questions immediately arise:

1. **Existence:** Does the right function — the one that solves our problem — even live inside our chosen hypothesis space?
2. **Feasibility:** If it does, can we find it with a reasonable amount of computation?

These two questions will drive the next two lectures.

Preview.

- Lecture 2 answers the *existence* question: for a sufficiently rich hypothesis space, any reasonable function can be approximated arbitrarily well.
- Lecture 3 answers the *feasibility* question: how do we actually search the parameter space?

1.4 From the Physical World to Raw Data

Before any learning can happen, we must collect data from the physical world.

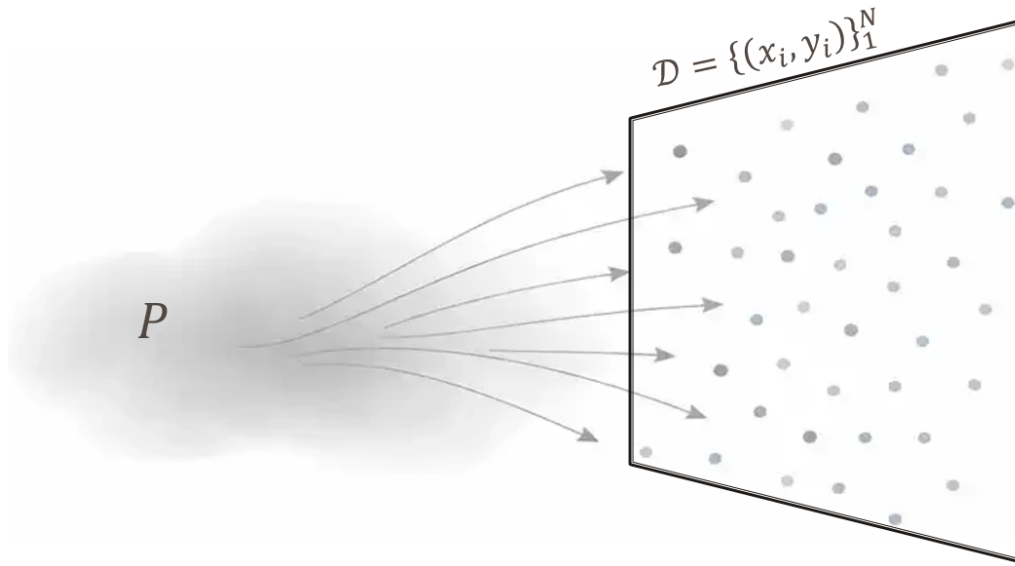


Figure 2: Continuous physical world $\mathcal{W}$ projected onto a discrete observation plane $\mathcal{D}$.

Let $\mathcal{W}$ represent the continuous, infinite-dimensional physical reality. Computers cannot process infinite-dimensional systems directly. Through sensors, cameras, or measurement devices, an observation mapping $\mathcal{O}$ discretizes reality into a finite dataset $\mathcal{D}$:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y} \quad (8)$$

Where:

- $x_i \in \mathbb{R}^D$ is a D -dimensional feature vector of raw measurements.
- $y_i \in \mathcal{Y}$ is the corresponding target label or value.
- N is the number of observations.

1.4.1 Data Is Always Imperfect

Physical sampling is constrained by sensor precision, unobserved variables, and environmental noise. Our observations are a corrupted version of the true relationship:

$$\tilde{f}(x) = f(x) + \epsilon \quad (9)$$

where $f(x)$ is the true underlying relationship, and ϵ is noise.

Important. We never see $f(x)$ directly. We only see noisy samples.

1.4.2 A Concrete Example

A digital photograph is a discretization of a continuous scene — a grid of pixels, each with a finite intensity. The physical world has been compressed into a raw data point. Learning happens on these discrete points, not on continuous reality.

1.5 The Core Pipeline: Observation $\rightarrow$ Representation

A primary goal of machine learning is to transform messy, entangled raw data into spaces where patterns become simple.

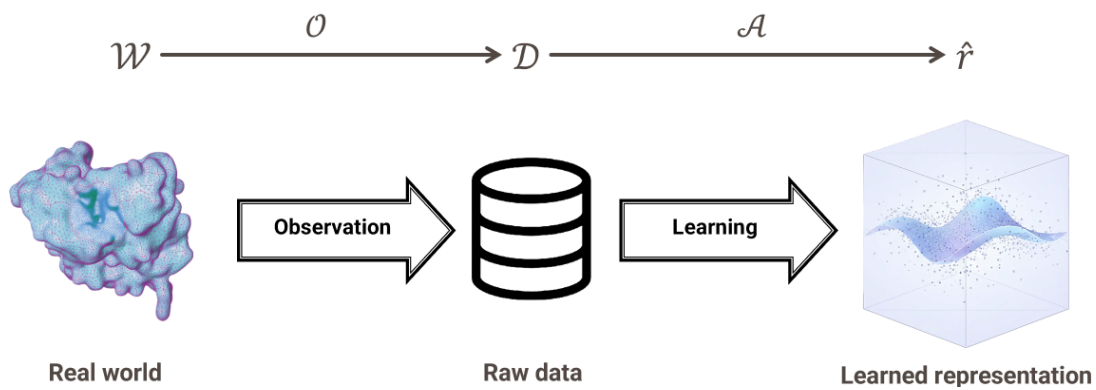


Figure 3: The complete observation and representation learning pipeline ($\mathcal{W} \xrightarrow{\mathcal{O}} \mathcal{D} \xrightarrow{\mathcal{A}} \hat{\mathcal{r}}$).

The process is a two-stage mapping pipeline:

1. **Physical Observation Mapping ($\mathcal{O}$):** The environment $\mathcal{W}$ is sampled to form the raw dataset $\mathcal{D}$.

$$\mathcal{O} : \mathcal{W} \longrightarrow \mathcal{D} \quad (10)$$

2. **Representation Mapping ($\mathcal{A}$):** A learning algorithm maps raw features into a learned secondary representation space $\hat{\mathcal{r}}$.

$$\mathcal{A} : \mathcal{D} \longrightarrow \hat{\mathcal{r}} \quad (11)$$

1.5.1 Why Do We Need a Secondary Representation?

In raw data spaces, decision boundaries between classes are often non-linear and entangled. The task of a learning algorithm is to construct a mapping $\phi : \mathcal{X} \rightarrow \hat{\mathcal{r}}$ such that entangled raw points become linearly separable, clustered, or geometrically simplified in $\hat{\mathcal{r}}$.

Intuition. Raw data is like a crumpled piece of paper with a drawing on it. A good representation is like smoothing out the paper so the drawing becomes easy to read.

1.6 Geometric Intuition: Why Representations Matter

Consider three classes of points arranged in three concentric rings below.

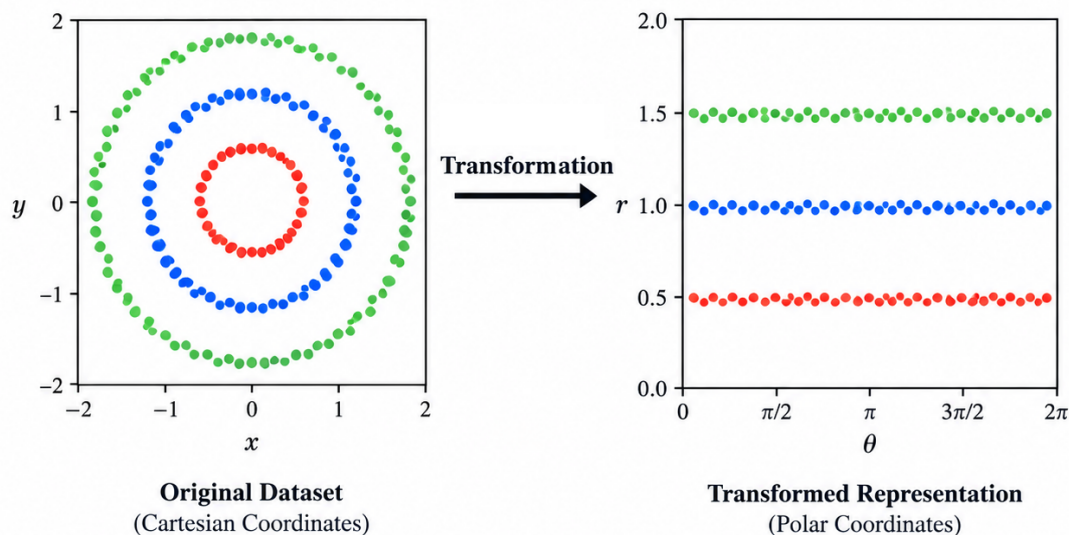


Figure 4: Cartesian space $\mathcal{D}$ (entangled concentric rings) versus polar space $\hat{r}$ (linearly separable bands).

The Problem in Raw Space

Class red consists of points near the origin. Class blue consists of points farther out and class green consists of points more further. In raw Cartesian coordinates (x_1, x_2) : The three classes are *not* linearly separable.

The Solution: A Secondary Representation.

Apply a polar transformation:

$$r = \sqrt{x_1^2 + x_2^2}, \quad \theta = \arctan2(x_2, x_1) \quad (12)$$

In the new representation space (as shown in figure 4) a simple linear threshold on r separates the classes perfectly.

Key Insight. The data did not change. Only the representation changed. By choosing the right space, a hard problem became trivial.

1.7 Practical Demonstration: Space Transformation in Python

The full, runnable notebook for this demonstration is maintained on GitHub. You can scan the code below or follow the link to open it directly.



Notebook: a simple space transformation

Takeaway. The same data can be hard or easy to learn from depending on its representation. A good representation does not just change the data — it reveals structure that was always there.

1.8 A Short Taxonomy of Learning Paradigms

Depending on how observations are structured, machine learning problems fall into four broad categories. We will explore each in depth later in the course. For now, just the names and the intuition:

Paradigm	Data	Goal	Example
Supervised	Labeled pairs (x, y)	Predict y from x	Spam classification
Unsupervised	Unlabeled x	Find hidden structure	Customer segmentation
Self-supervised	Unlabeled x + proxy task	Learn representations	Masked language modeling
Reinforcement	Interaction + rewards	Maximize cumulative reward	Game playing

Table 2: A short taxonomy of learning paradigms.

Note. We will not use the math of these paradigms today. The point is just to know that “machine learning” is not one thing — it is a family of problem settings that share the same optimization-based core.

1.9 Summary and Key Takeaways

1. **The Engineering Shift.** Classical software engineering encodes human knowledge into hand-written rules. Machine learning replaces this with empirical parameter search over a family of candidate functions.
2. **The Hypothesis Space.** The choice of $\mathcal{F}$ determines both the expressiveness of our model and the difficulty of searching it.
3. **The Data Pipeline.** Physical reality $\mathcal{W}$ is sampled into a dataset $\mathcal{D}$ via $\mathcal{O}$. A learning algorithm $\mathcal{A}$ then transforms $\mathcal{D}$ into a representation space $\hat{r}$ where patterns become simple.
4. **Representation Matters.** The same data can be hard or easy to learn from depending on its representation.

Two Big Open Questions:

- Does the right function exist in our hypothesis space? → Lecture 2.
- Can we find it? → Lecture 3.

1.10 Before the Next Lecture

Think about this question: *Suppose we choose a hypothesis space that is so rich it can represent any function. Does that guarantee we can solve the problem? What could still go wrong?*